PREPRINT · ARXIV 2609.27291 · 2026

NGN: Learning Neural Network Size as a Differentiable Count

Lixing Li · Cornell University

The Neurogenesis Network (NGN) turns a structural size, such as a width, head count, depth, or LoRA rank, into **one learnable boundary** over an ordered sequence of components, trained jointly with the weights. The boundary can grow from a compact start, and components beyond it are simply discarded at deployment.

MLP CNN GNN Transformer Mamba LoRA Adapters Depth Structured pruning

MOTIVATION

Model size is usually chosen before training starts

1 THE PROBLEM

- Widths, depths, and other sizes are **fixed before the weights are learned**.
- Picking them takes expertise or **repeated training** across candidate sizes.
- Useful capacity depends on the data and the objective, so size could instead be **optimized alongside the weights**.

2 HOW PRIOR WORK PICKS SIZE

`Pruning` Scores and removes parts of an already trained model (SparseGPT, Wanda).

`Sparsify` Learns which parts of a fixed, overcomplete model survive, with one gate per part (L_0, SoRA).

`Flexible` Supports many deployment sizes, or varies compute per input (DyLoRA, ACT).

`Grow` Changes active capacity during training (UDN over depth, AWNN over width).

3 NGN'S IDEA

- Learn **how many** ordered components to use, as one count per component group.
- One shared boundary, not many independent survival gates.
- The formulation sets **no maximum budget**: new components can be added as the boundary advances.

CONTRIBUTIONS

1 A parameterization that represents size as a differentiable count over ordered components, learned jointly with the weights.

2 Controlled analyses of when the learned count is stable and deployable, and how it compares with fixed sizes and other capacity learners.

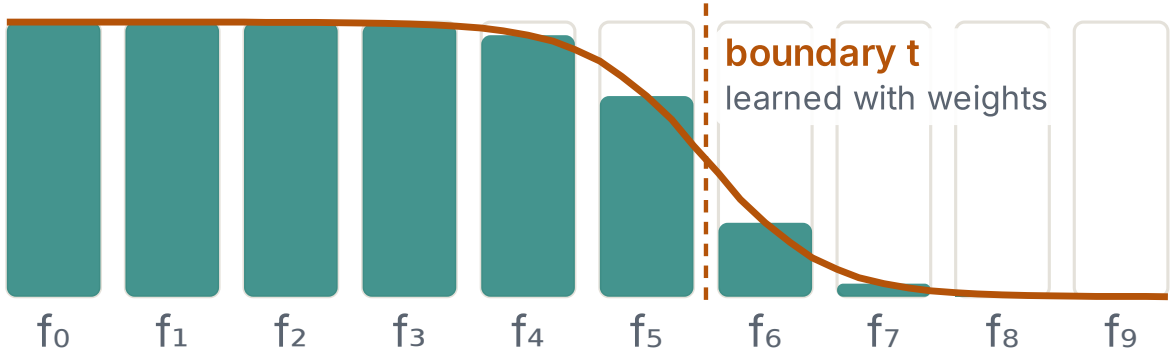
3 The same mechanism across model families, tasks, and structural axes, including different allocations per layer.

One boundary t gates an ordered prefix of components

SCHEMATIC · ILLUSTRATIVE SIZES, NOT DATA

TRAINING · SOFT COUNT GATE

$$y = \Sigma\ w_k\ g_k(t)\ f_k(x)$$
$$g_k(t) = \sigma(\beta(t - k + \delta))$$



$J = L + \lambda t + \alpha \sin^2(\pi t)$
λt = capacity price

anneal $\beta \rightarrow$ sharp gate
$\alpha \sin^2(\pi t)$ snaps t to an integer

DEPLOYMENT · HARD TRUNCATION



HOW THE BOUNDARY MOVES

$$J = L_{task} + \lambda t + \alpha \sin^2(\pi t)$$

- **Capacity price** λt pushes the boundary inward.
- **Task gradient** is concentrated at the frontier, so a useful next component **recruits** it outward.
- **Snapping** $\alpha \sin^2(\pi t)$ has minima at integers; λ is annealed to zero so they stay exact.
- **Sharpening** β (e.g. 4 → 12): soft early so gradients reach the frontier, sharp late so the hard prefix matches.
- Weights $w_k = 2^{-k}$ order the candidates; f_k are zero-initialized.

Needs nested capacity: each added f_K must strictly enlarge what the prefix can fit.

On a 1D regression task, the count settles and truncates cleanly

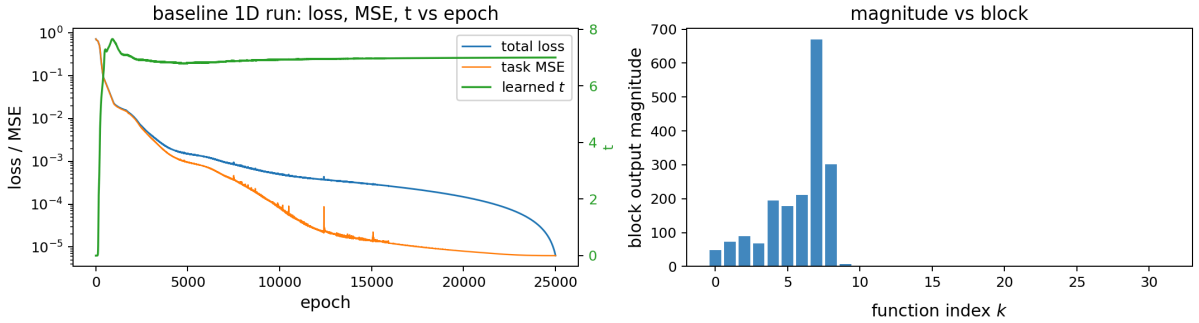


Fig. 1: baseline run. *t* grows fast, overshoots, then settles; blocks past the boundary stay near zero.

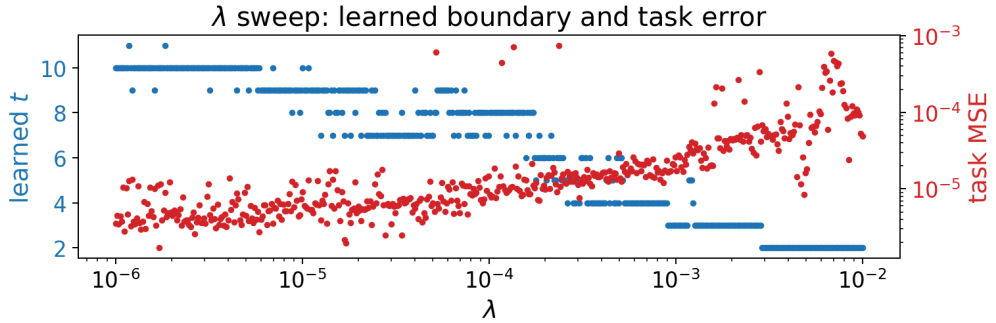


Fig. 2: λ from 10^{-6} to 10^{-2}, 500 models. Higher price gives a smaller count, in integer plateaus.

7.00

final boundary *t* in the baseline run (Fig. 1)

6.27e-06 vs **6.28e-06**

soft MSE vs truncated MSE: discarding the tail barely changes the fit

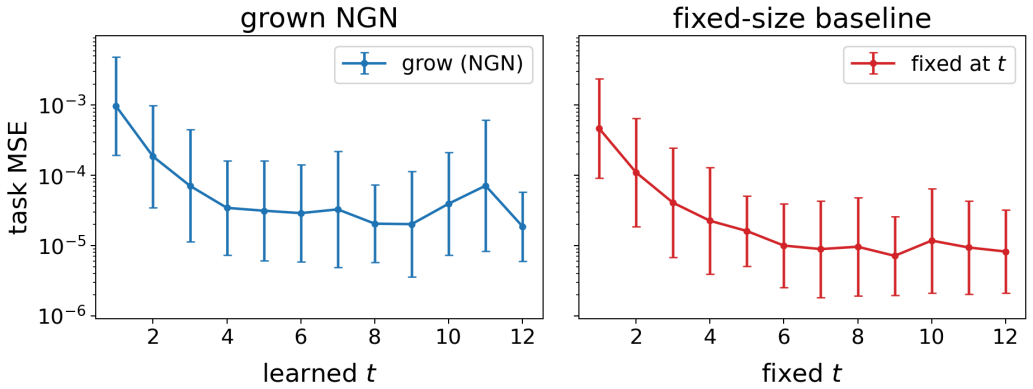
SCHEDULE ABLATION (TABLE 1)

SETTING	SOFT MSE	TRUNC.
baseline	**1.31e-05**	**1.31e-05**
α = 0	5.43e-05	4.83e-04
β = 4 const	1.50e-05	6.01e-04

Medians over 10 seeds. Removing snapping or sharpening opens a soft-to-truncated gap.

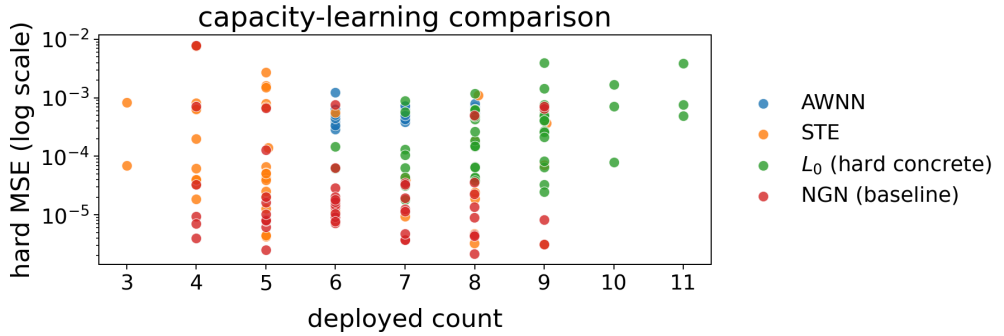
Versus fixed sizes and other ways of learning capacity

GROWN VS FIXED AT THE SAME COUNT



Over 50 seeds, both trace nearly the same error–count curve, and fixed models are **typically slightly better** at a matched count. The benefit is **automatic size selection**, not a better model at that size.

VERSUS OTHER CAPACITY LEARNERS



- **L_0** (hard concrete) generally sits at higher error.
- **AWNN** picks a more stable count across seeds but fits worse.
- **STE** (hard forward gate) gets stuck at small counts more often.

Fig. 4: 50 seeds, one point per seed.

The same boundary works across model families

TASK	WHAT F_K IS	LEARNED T	SOFT	TRUNC.	FIXED	PRUNES BACK?
MNIST	tanh MLP features	[8.00]	96.93	96.93	96.81	yes
MNIST	conv branches	[4.00]	99.18	99.18	99.13	partial
CIFAR-10	conv branches	[7.02]	71.77	71.77	72.03	partial
Cora	2-hop GCN branches	[1.00]	79.05	79.05	79.45	partial
TinyStories	attention heads (2 layers)	[2.11, 3.03]	1.3055	1.3234	1.3011	no
TinyStories	Mamba state dim. N (2 layers)	[1.00, 2.00]	1.3551	1.3597	1.3445	partial

READING TABLE 3

Accuracy % (higher is better) for the first four rows, medians of 10 seeds; TinyStories bits/char (lower is better), single seed. **Truncation usually costs little**, and matched fixed retraining performs similarly. Pruning back from high capacity is **not reliable**, so the claim covers growth and truncation only. FFN and joint head+FFN rows are omitted here.

LEARNING DEPTH (TABLE 4)

A **recursive** chain with one global boundary learns depth 8 (t = 7.02); soft / truncated / fixed MSE 2.49e-05 / 3.56e-05 / 1.54e-05.

Per-layer boundaries over ten residual layers learn a clean depth of six only with both proposed schedules (λ decreasing and r increasing across layers).

Learning LoRA rank and adapter width on Llama-3-8B-Instruct

LORA RANK (TABLE 5)

TASK	BASE	NGN RANK (E/M/L)	NGN SOFT / TRUNC.	FIXED	SORA	ADALORA
WikiSQL-easy	**3.5**	**19 (2/9/8)**	**78.2 / 78.6**	**77.7**	**75.2**	**45.1**
WikiSQL-hard	2.0	21 (4/11/6)	70.8 / 70.4	68.7	67.4	45.4
Spider	1.1	17 (5/7/5)	12.6 / 12.4	12.3	13.9	10.3
GSM8K	45.2	18 (7/8/3)	53.6 / 55.2	50.8	35.6	56.8
SQuAD	65.2	13 (2/7/4)	67.2 / 67.3	67.1	67.1	66.9

Single seed per configuration. e/m/l splits the learned rank across early / middle / late thirds of the layers. SoRA and AdaLoRA target NGN's learned rank; SoRA ends at a lower rank on Spider and GSM8K.

The same mechanism also learns **adapter width** (nonlinear bottlenecks): for example, width 4 on WikiSQL-easy, soft / truncated 83.2 / 80.5 vs fixed 82.4.

HIGHLIGHTS

78.6 vs 75.2 / 45.1
WikiSQL-easy, truncated NGN LoRA at learned rank 19, vs SoRA / AdaLoRA at the same rank

NGN scores higher than **SoRA on all three** exact rank matches, and higher than **AdaLoRA on four of five** tasks.

Learned ranks differ by layer; for both WikiSQL variants they concentrate in the **middle third**.

Both baselines need a finite maximum rank, and AdaLoRA also a target rank. NGN learns one ordered boundary per layer instead.

62.5 vs 17.0 / 69.5
Direct FFN pruning at 14.1% kept (appendix Table 15): NGN vs random drop / Wanda exact match. Far above random; Wanda is slightly ahead.

Limitations and takeaways

LIMITATIONS

- **Ordered candidates only.** Components must form a nested sequence; the boundary cannot skip a harmful early one to reach a useful later one.
- **Finite growth range.** Because $w_k = 0.5^k$ decays exponentially, high-index components are hard to recruit.
- **Optimization cost.** It takes more steps than a fixed model; with the same step budget it is often similar or slightly worse.
- **Still needs tuning.** λ sets the size–performance trade-off; too little snapping pressure from α weakens hard truncation.
- **Pruning back is not reliable.** Starting from high capacity does not consistently return to the grown size outside the controlled task.

TAKEAWAYS

- Structural capacity can be **optimized directly as a count**: one boundary, a capacity price, an integer snap, and gate sharpening.
- The same parameterization covers feature functions, conv and graph branches, attention heads, Mamba state, residual depth, LoRA, and adapters.
- Deploying only the learned prefix **usually changes performance little**, and the chosen sizes perform similarly to fixed models trained at that size.
- The benefit is **automatic size selection**. The learned size depends on the task objective and the chosen price.
- Count learning is most useful when capacity is **naturally ordered**.